

RESEARCH

AI-Assisted Design-to-Code

Figma Make · MCP Server · Claude Code · Angular 18

Ginés López Montalbán

Product Designer — UX/UI & Frontend Development

July 2026

Figma Make	MCP Server	Claude Code	Angular 18	Design Systems	Semantic Tokens	Living Spec	AI Workflow	Component Architecture
------------	------------	-------------	------------	----------------	-----------------	-------------	-------------	------------------------

Overview

A structured two-day research into AI-assisted workflows connecting three distinct technological layers — Figma Make, the Figma MCP Server, and Claude Code — to generate production-ready Angular components and views from design specifications. The exercise used a medical blood-test results screen as a practical scenario, covering the complete pipeline from design system definition to Angular 18 implementation. Beyond the immediate deliverables, the research produced a reusable methodology: context documents, prompt libraries, component specification templates, and a set of empirically-derived rules for future AI-assisted development projects.

Research Objectives

Evaluate the Figma AI ecosystem	Assess the practical capabilities and limitations of Figma Make, the MCP Server, and their integration with Claude Code for frontend development workflows.
Test the design-to-code pipeline	Execute and document the full chain from design system creation in Figma Make to Angular 18 component generation via Claude Code and MCP.
Define a reusable methodology	Produce structured context documents, prompt templates, and component specification standards that can be applied to future projects without repeating the discovery process.
Identify gaps and improvement areas	Document friction points, anti-patterns, and architectural decisions that emerge empirically — turning problems into actionable rules for the next iteration.

Establish inter-layer responsibilities

Define clear boundaries between Claude.ai (orchestration), Figma Make (design & spec), and Claude Code (implementation) to avoid ambiguity and overlapping responsibilities.

Methodology

Phase 0	Style guide	Color, typography and spacing tokens defined as visual reference frames in Figma Make.
Phase 1	Components	Three components built with living spec headers: InputText, Button, CardResult with clinical state variants.
Phase 1.5	Spacing tokens	Spacing scale applied to all components before building views.
Phase 2	Views	Two responsive screen variants generated using component instances, not redrawn divs.
Phase 2.5	Review & adjust	Direct code editing and prompt-based refinement in the Make sandbox before export.
Phase 3	Export to Angular	Two-phase export via Claude Code + MCP: analysis with warnings, then generation after confirmation.
Phase 3C	Final verification	Compilation, import checks, SCSS token validation, and visual render verification.

Key Findings

- 01

Figma Make generates rendering code, not production code

Components use inline styles, literal props (hex colors, opacity values), and non-semantic HTML elements like divs instead of buttons or inputs. This is expected behaviour for a prototyping sandbox, not a failure — but it requires a structured translation strategy to Angular.
- 02

The living spec header is the critical bridge

Defining a structured technical specification header (inputs, variants, tokens, layout, HTML semantics) directly in the Figma Make canvas before the visual representation proved to be the most effective mechanism for guiding Claude Code toward correct Angular output. The header takes priority over the visual and over the React code.
- 03

The two-phase export strategy reduces errors significantly

Separating analysis (with explicit warning reporting) from generation — and requiring confirmation between phases — caught 12 warnings across components and views before a single line of Angular code was written. A warning detected before generation costs one message; the same problem after generation can cost an hour of refactoring.
- 04

Semantic tokens are non-negotiable

Components that receive color as instance data (accent, bg, border as hex strings) create translation complexity that does not exist if the component uses semantic variants from the start. The rule 'colors are never @Input() values' must be enforced as a design constraint, not a code review.

05 **Figma Make does not instantiate components in views — it redraws them**

Views generated in Make used generic divs visually resembling the defined components, not actual component instances. This breaks the composition chain and generates layout ambiguity in the export. Explicit prompt instructions are required to force Claude Code to import existing components rather than regenerate them.

06 **The real power of the Figma ecosystem lies in a different chain**

The research revealed that Figma Make is a rapid prototyping tool, not a design-to-code bridge. The chain with the highest potential for Angular development is: Figma Design (structured Variables & components) → MCP → Code Connect → Claude Code. Code Connect maps Figma component instances directly to Angular components with their correct imports and prop interfaces, eliminating the translation problem entirely.

Conclusions

- Figma Make accelerates visual iteration significantly, but requires structured discipline (living spec headers, semantic token architecture, checklist verification) to produce output that Claude Code can translate to Angular without manual intervention.
- The two-phase export strategy (analysis → confirmation → generation) is a practical quality gate that should be standard in any AI-assisted code generation workflow, regardless of the design tool used.
- The ambiguity principle applies universally: anything not explicitly specified will be decided by the AI tool using its own criteria. The quality of the output is directly proportional to the specificity of the input context.
- The research produced a reusable methodology: five structured documents covering system responsibilities, Claude.ai orchestration context, project kickstart checklist, Figma Make context template, and Claude Code context template. These form the foundation for applying AI-assisted development systematically to future projects.
- The next logical step — Figma Design with Code Connect — would eliminate the translation problem at its root by exposing structured component specifications (typed properties, semantic variants, token bindings) directly to Claude Code via MCP, rather than React rendering code that requires interpretation.

Research Deliverables

Evaluation report	35-page document covering the full exercise: objectives, layer descriptions, prompts, execution results, warnings, and strategic conclusions.
figma_make_context.md	Reusable context document for Figma Make projects, including token architecture rules, component construction order, living spec template, and anti-pattern checklist. Includes v1.1 corrections derived from the exercise.
DOC 01 — System responsibilities	Definition of the three-layer architecture (Claude.ai / Figma Make / Claude Code), their responsibilities, what each layer needs to know, and the inter-layer authority hierarchy.

DOC 02 — Claude.ai initial context	Orchestration instructions and six-block project kickstart checklist, ready to load as context in any new project.
Angular project	Functional Angular 18 application with standalone components (InputText, Button, CardResult), responsive results view, semantic token architecture in SCSS, and lazy-loaded routing — generated via Claude Code + MCP from the Figma Make file.

This research demonstrates that AI-assisted design-to-code workflows are viable today, and that their quality depends not on the tools themselves but on the discipline of eliminating ambiguity at every layer of the process.